

Fox Glacier Feature Tracking Lab

This lab runs the full Fox Glacier feature tracking workflow in GitHub Codespaces. You will run three notebooks to generate the research-area map, check and download Sentinel-2 optical images, calculate glacier surface motion with autoRIFT, and then use a Streamlit viewer to inspect the final result.

1. Open GitHub Codespaces

1. Open GitHub Codespaces in a browser and sign in with your GitHub account:

```
https://github.com/codespaces
```

2. Click **New codespace**.
3. In **Select a repository**, search for and select:

```
GuoXin-Ink/Remote-Sensing-Application
```

Tip: A 4-core Codespace will run faster than the smallest option.

4. Click **Create codespace**.
5. Wait for the Codespace environment to finish building. The first setup installs the geospatial environment and may take about 8 min. You can check the progress from the lower-right status area.

```
Preparing transaction: done
Verifying transaction: done
Executing transaction: /
-
done
#
# To activate this environment, use
#
#   $ conda activate rsa-lab
#
# To deactivate an active environment, use
#
#   $ conda deactivate

Registering Jupyter kernel: Python (rsa-lab)
Installed kernelspec rsa-lab in /home/vscode/.local/share/jupyter/kernels/rsa-lab
Activating rsa-lab automatically in new terminals
Codespaces setup complete. Select the Jupyter kernel: Python (rsa-lab).
Outcome: success User: vscode WorkspaceFolder: /workspaces/Remote-Sensing-Application
devcontainer process exited with exit code 0
Configuring codespace...
Running commands...
$ devcontainer up --id-label Type=codespaces --workspace-folder /var/lib/docker/codespacemount/workspace/Remote-Sensing-Application --expect-existing-container --skip-post-attach --mount type=bind,source=/.codespaces/agent/mount/cache,target=/vscode --container-data-folder .vscode-remote/data/Machine --container-system-data-folder /var/vscode-remote --log-level trace --log-format json --update-remote-user-uid-default never --mount-workspace-git-root false --config "/var/lib/docker/codespacemount/workspace/Remote-Sensing-Application/.devcontainer/devcontainer.json" --override-config /root/.codespaces/shared/merged_devcontainer.json --default-user-env-probe loginInteractiveShell --container-session-data-folder /workspaces/.codespaces/.persistedshare/devcontainers-cli/cache --secrets-file /root/.codespaces/shared/user-secrets-envs.json
[200 ms] @devcontainers/cli 0.83.3. Node.js v18.20.8. linux 6.8.0-1044-azure x64.
Outcome: success User: vscode WorkspaceFolder: /workspaces/Remote-Sensing-Application
```

6. After the Codespace connects, open the **FeatureTrackingLab/** folder.
7. For each notebook, choose the Jupyter kernel:

```
Python (rsa-lab)
```

The kernel is created from `.devcontainer/environment.yml`. If `Python (rsa-lab)` is not visible yet, wait until the setup finishes and refresh the page.

2. Run Order

Run the files in this order:

1. `plot_south_island_dem.ipynb`
2. `optical_image_check.ipynb`
3. `fox_glacier_b08_autorift.ipynb`
4. `run_streamlit_viewer.ipynb`

Main fixed input files:

- `fox_glacier_from_osm.geojson`: Fox Glacier boundary.
- `earth_relief_30s_south_island.nc`: South Island DEM subset used in Part 1.
- `4. autorift_speed_scatter_app.py`: Streamlit app used by Part 4.

All generated outputs are saved in the current `FeatureTrackingLab/` folder.

3. Part 1: Research Area Map

Notebook:

1. `plot_south_island_dem.ipynb`

Open the notebook and click `Run All`. The notebook usually takes about 30 s.

Goal:

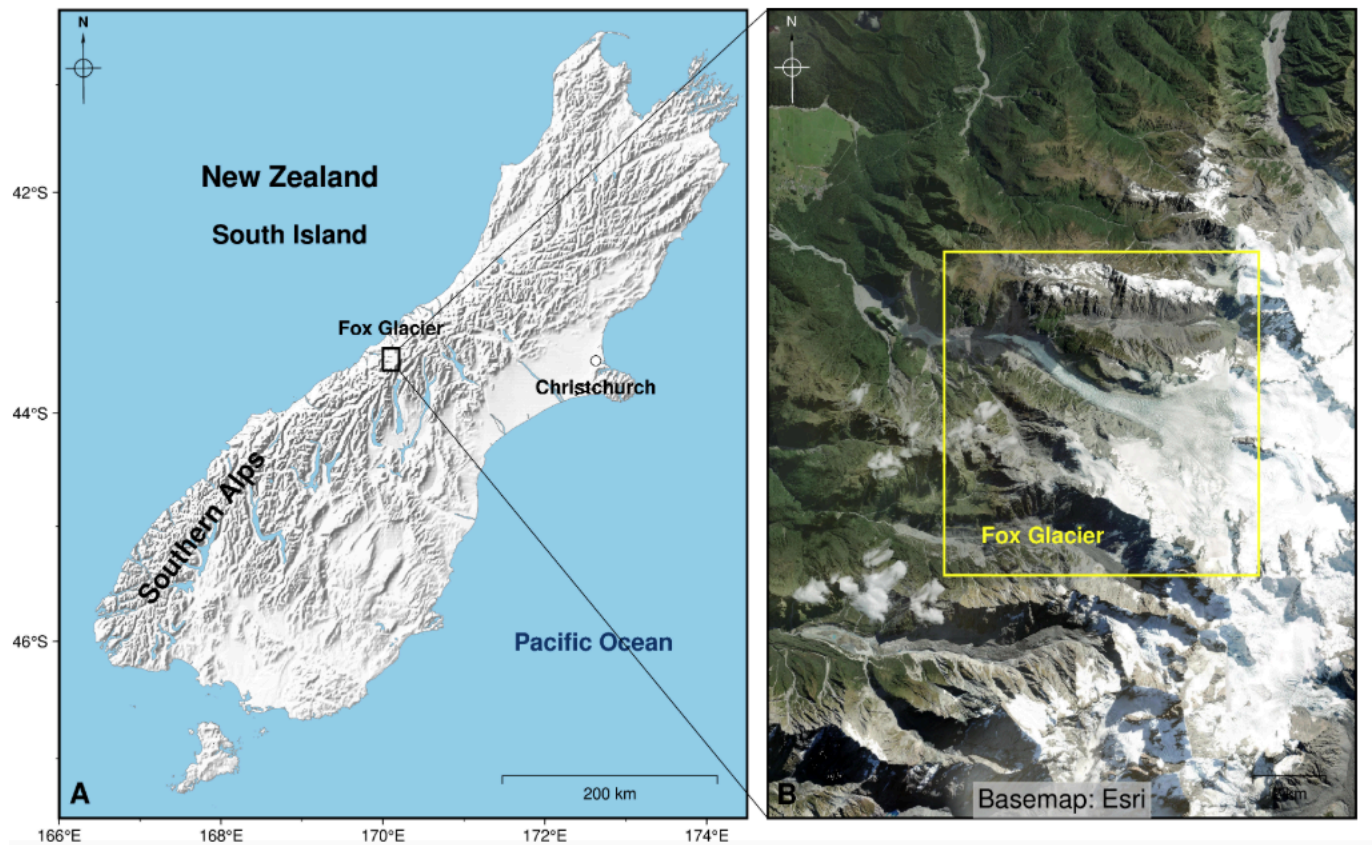
- Draw the South Island overview map.
- Draw the Fox Glacier inset map.
- Save the output figure:

`south_island_dem.png`

Workflow:

1. Load the local DEM subset `earth_relief_30s_south_island.nc`.
2. Use PyGMT to draw the South Island terrain map.
3. Use Contextily and GDAL to create the Fox Glacier local basemap.
4. Add the glacier bounding box, city labels, scale bars, north arrow, panel labels, and connecting lines.
5. Save the final PNG.

Open the generated PNG from the file list on the left.



Adjustable parameters:

Parameter	Current value	Meaning
R_LEFT	[166.0, 174.5, -47.5, -40.3]	South Island overview extent: west, east, south, north
FOX_W, FOX_E, FOX_S, FOX_N	170.0, 170.2, -43.62, -43.42	Fox Glacier inset extent
TILE_ZOOM	14	Basemap zoom level; higher values give more detail but need more processing time
GAP_CM	0.45	Gap between the left and right panels
LEFT_FONT_ANNOT	"10p"	Axis annotation font size for the overview map
LABEL_PANEL	"18p, Helvetica-Bold, black"	Panel label font
LEFT_MAP_SCALE	"jBR+w200k+o0.65c/0.75c"	Scale bar setting for the overview map
RIGHT_MAP_SCALE	"jBR+w2k+o0.65c/0.75c"	Scale bar setting for the inset map

Font-size settings:

```
fig.text(..., font="12p,Helvetica-Bold,black")
LABEL_PANEL = "18p,Helvetica-Bold,black"
LEFT_FONT_ANNOT = "10p"
```

Map-extent settings:

```
R_LEFT = [166.0, 174.5, -47.5, -40.3]
FOX_W, FOX_E, FOX_S, FOX_N = 170.0, 170.2, -43.62, -43.42
```

4. Part 2: Optical Image Check and Download

Notebook:

```
2. optical_image_check.ipynb
```

Choose the **Python (rsa-lab)** kernel and click **Run All**. This notebook searches Sentinel-2 L2A images from 2022 to 2026 over Fox Glacier, keeps scenes with AOI cloud cover no higher than 10% and AOI snow cover no higher than 40%, and downloads selected B08 NIR images. It usually takes about 30 s.

Goal:

- Search Sentinel-2 L2A images from Microsoft Planetary Computer.
- Use the Sentinel-2 SCL band to calculate cloud and snow percentages inside the AOI.
- Filter images using the cloud and snow thresholds.
- Save quicklook figures.
- Download selected B08 near-infrared GeoTIFFs for Part 3.

Outputs:

```
fox_glacier_b08_quicklook.png
fox_glacier_b08_quicklook_dates.png
B08_fox_2026-03-01.tif
B08_fox_2026-03-21.tif
```

Workflow:

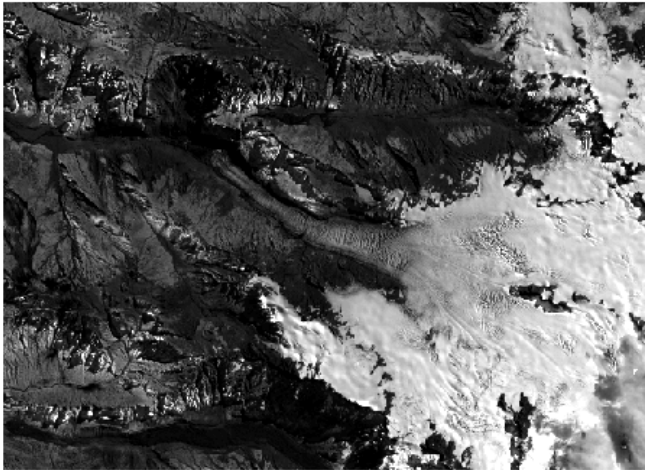
1. Load **fox_glacier_from_osm.geojson**.
2. Build a buffered AOI bounding box around the glacier boundary.
3. Search Sentinel-2 L2A scenes within the date range.
4. Read the SCL chip for each scene.
5. Calculate AOI cloud and snow percentages.
6. Remove scenes above the cloud or snow threshold.
7. Create quicklook figures for the accepted scenes.

8. Download B08 GeoTIFFs for the dates listed in [DOWNLOAD_DATES](#).

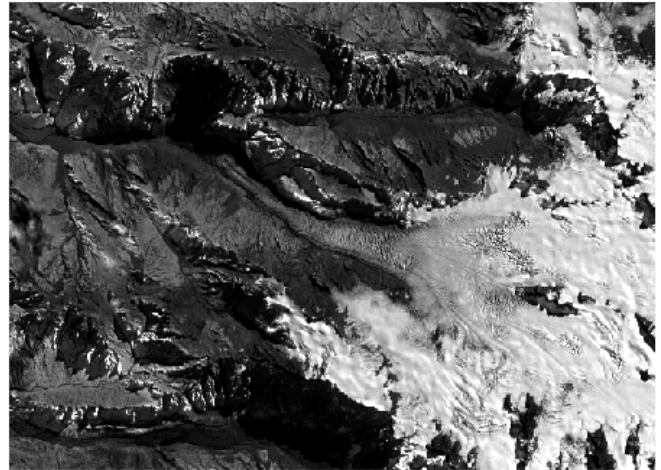
Open the generated PNG files from the file list on the left.

S2 B08 selected dates (Fox crop)

B08_fox_2026-03-01



B08_fox_2026-03-21



Main parameters:

Parameter	Current value	Meaning
PRESET_DATETIME	2022-01-01T00:00:00Z/2026-12-31T23:59:59Z	STAC search date range
DOWNLOAD_DATES	["20260301", "20260321"]	Dates downloaded for feature tracking
PRESET_BUFFER_DEG	0.02	AOI bbox buffer in degrees
PRESET_SCL_CLOUD_CLASSES	(3, 8, 9, 10)	SCL classes treated as cloud, cloud shadow, or cirrus
PRESET_SCL_SNOW_CLASSES	(11,)	SCL class treated as snow/ice
PRESET_MAX_AOI_CLOUD_PCT	10.0	Maximum allowed AOI cloud percentage
PRESET_MAX_AOI_SNOW_PCT	40.0	Maximum allowed AOI snow percentage

Change the search date range:

```
PRESET_DATETIME = "2022-01-01T00:00:00Z/2026-12-31T23:59:59Z"
```

Change the download dates:

```
DOWNLOAD_DATES = ["20260301", "20260321"]
```

The dates in `DOWNLOAD_DATES` should be inside `PRESET_DATETIME`, and the corresponding Sentinel-2 scenes must pass the cloud and snow filters.

Change the cloud and snow thresholds:

```
PRESET_MAX_AOI_CLOUD_PCT = 10.0  
PRESET_MAX_AOI_SNOW_PCT = 40.0
```

Lower thresholds are stricter and usually leave fewer usable dates. Higher thresholds allow more dates but may include lower-quality images.

5. Part 3: autoRIFT Feature Tracking

Notebook:

```
3. fox_glacier_b08_autorift.ipynb
```

Open the notebook and click `Run All`. The notebook usually takes about 10 s after the B08 files have been downloaded.

Goal:

- Read the two B08 GeoTIFFs downloaded in Part 2.
- Coregister the overlapping area of the two images.
- Run autoRIFT optical feature tracking.
- Convert pixel displacement to velocity in m/day.
- Save static result figures and the `.npz` file used by Streamlit.

Inputs:

```
B08_fox_2026-03-01.tif  
B08_fox_2026-03-21.tif  
fox_glacier_from_osm.geojson
```

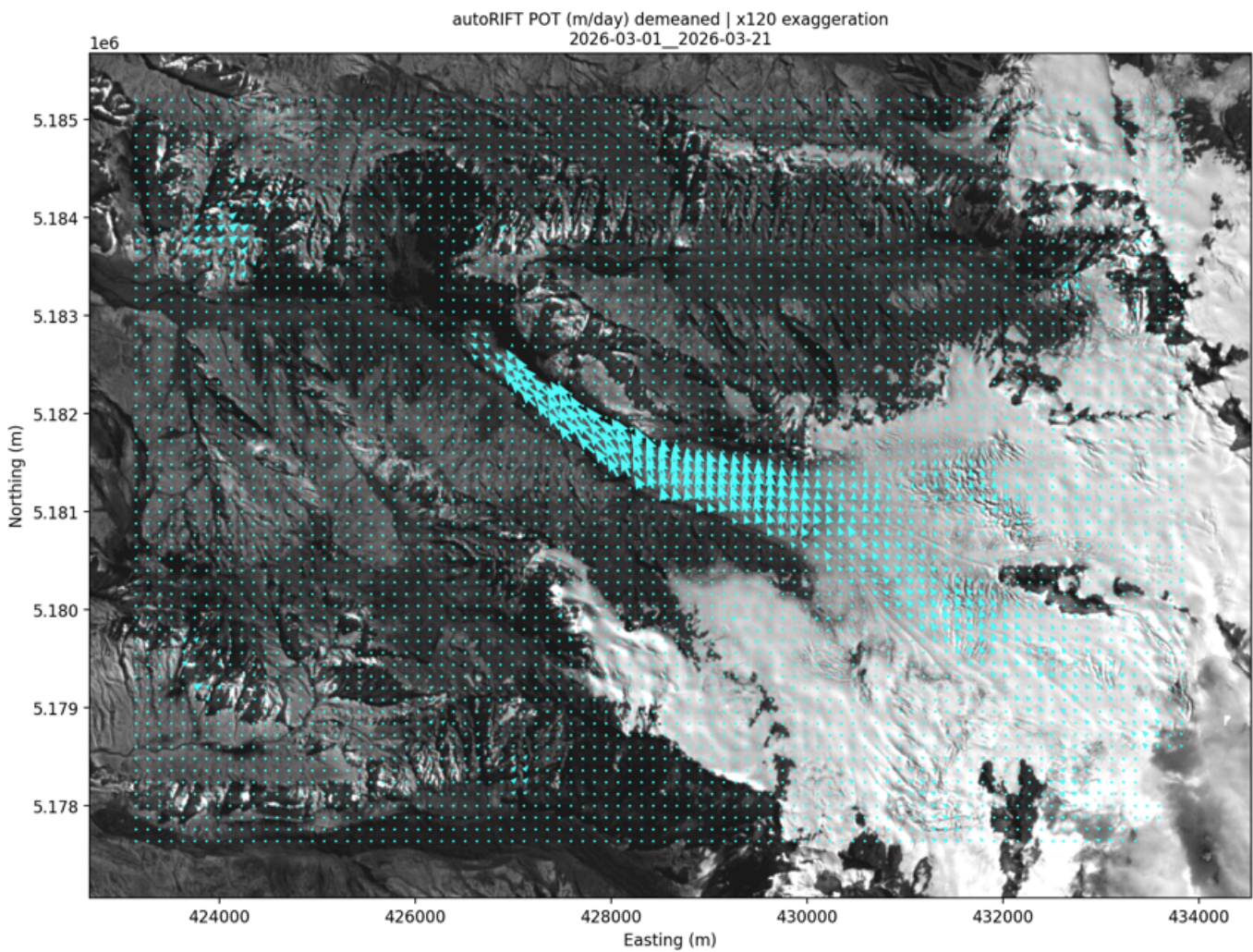
Outputs:

```
autorift_scatter_demeaned_2026-03-01__2026-03-21.npz  
autorift_pot_quiver_demeaned_2026-03-01__2026-03-21.png  
autorift_pot_speed_scatter_demeaned_2026-03-01__2026-03-21.png  
autorift_pot_kde_demeaned_2026-03-01__2026-03-21.png
```

Workflow:

- 1. Load the reference image and secondary image.
- 2. Use `GeogridOptical().coregister(...)` to calculate the image overlap.
- 3. Build a regular tracking grid inside the overlap area.
- 4. Apply Wallis preprocessing to both images.
- 5. Run autoRIFT to estimate pixel displacement.
- 6. Use the date difference to convert displacement to m/day.
- 7. Subtract the `nanmedian` from `vx` and `vy` to get demeaned velocity.
- 8. Save the quiver figure, speed scatter figure, KDE figure, and `.npz` dataset.

Open the generated PNG files from the file list on the left. Example quiver result:



Main feature tracking parameters:

Parameter		Current value	Meaning
DEFAULT_REF	B08_fox_2026-03-01.tif		Reference image
DEFAULT_SEC	B08_fox_2026-03-21.tif		Secondary image
WALLIS_WIDTH		7	Wallis filter window size
SEARCH_LIMIT		72.0	Maximum search distance in pixels
GRID_STEP		12	Spacing between tracking chip centers

Parameter	Current value	Meaning
MARGIN	48	Pixels excluded near the image edge
threads	4	Number of parallel threads
exaggerate	120.0	Arrow exaggeration factor in the static quiver plot
dpi	160	Static figure output resolution

Parameter effects:

- Larger GRID_STEP: fewer points, faster run, sparser result.
- Smaller GRID_STEP: more points, slower run, denser result.
- Larger MARGIN: fewer edge mismatches, but a smaller usable area.
- Larger SEARCH_LIMIT: allows larger displacement, but may increase mismatches.
- Different WALLIS_WIDTH: changes local contrast enhancement before matching.
- More threads: can speed up processing if enough CPU resources are available.

6. Part 4: Streamlit Interactive Viewer

Notebook:

```
4. run_streamlit_viewer.ipynb
```

Streamlit app:

```
4. autorift_speed_scatter_app.py
```

Open the notebook, but do not click **Run All**. The last cell stops the Streamlit app, so run the cells one by one. After running the start Streamlit cell, open the forwarded **8501** link from the Codespaces **Ports** panel.

Tip: Use the Streamlit controls to adjust the velocity plot and quiver plot interactively.

Goal:

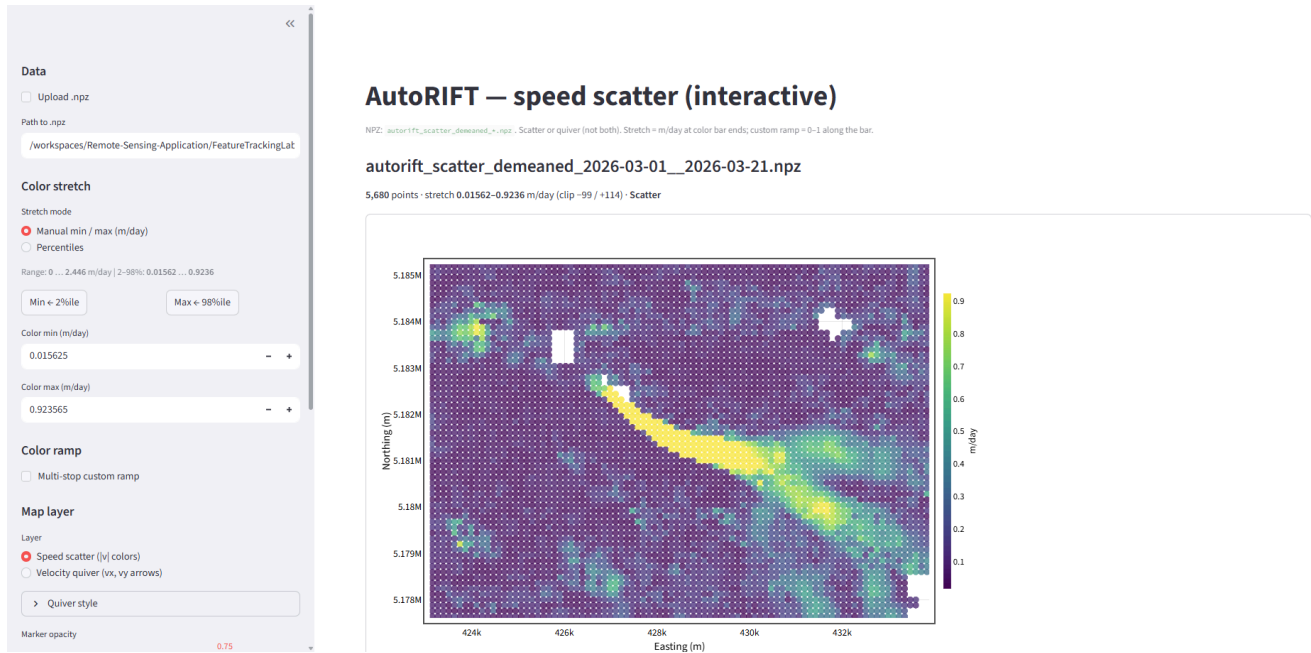
- Start the Streamlit viewer in Codespaces.
- Open the web interface through forwarded port **8501**.
- Inspect the autoRIFT **.npz** result interactively.

Run steps:

1. Open **4. run_streamlit_viewer.ipynb**.
2. Select the **Python (rsa-lab)** kernel.
3. Run the dependency check cell.
4. Run the file check cell.
5. Run the start Streamlit cell.

7. When finished, run the stop cell.

Viewer example:



Interactive Streamlit controls:

Control	Meaning
Stretch mode	Choose manual color limits or percentile-based color limits
Color min / max (m/day)	Set manual color scale limits
Color min / max (percentile)	Set color scale limits using percentiles
Multi-stop custom ramp	Build a custom multi-stop color ramp
Built-in colormap	Choose a built-in Plotly colormap
Layer	Switch between speed scatter and velocity quiver
Length x demeaned (m/day)	Adjust arrow length scaling
Arrowhead, Head angle, Line width, Color	Adjust arrow style
Marker opacity	Adjust scatter point opacity
Marker size	Adjust scatter point size
Show raster basemap under plot	Show the B08 GeoTIFF basemap below the result
Basemap opacity	Adjust basemap transparency

The Streamlit viewer uses the `.npz` file generated in Part 3. Feature tracking parameters should be changed in Part 3 before regenerating the `.npz` result.